

Module 3: Modern Angular Control Flow

Building Dynamic User Interfaces with Angular 21

ANGULAR 21 FRONT-END DEVELOPMENT

Module Overview

Learning Objectives

By the end of this module, you'll have a solid, practical understanding of Angular's modern control flow syntax — enabling you to write cleaner, more maintainable templates from day one.

1	**Understand Control Flow** Grasp the core philosophy behind Angular's modern declarative template syntax.
2	**Conditional Rendering** Implement `@if` and `@else` blocks for dynamic conditional UI.
3	**Collection Display** Render lists and collections efficiently using the `@for` block.
4	**Tracking & Branching** Optimize rendering with tracking strategies and handle multi-condition logic with `@switch`.
5	**Migration Path** Confidently migrate legacy structural directives to modern Angular standards.

Why Modern Control Flow?

Earlier Angular versions relied heavily on structural directives — `*ngIf`, `*ngFor`, and `*ngSwitch` — which required importing `CommonModule` and added boilerplate to every template. Angular 17+ introduced a first-class control flow syntax that lives directly in your templates, no imports needed.

Legacy Approach

- `*ngIf` structural directive
- `*ngFor` structural directive
- `*ngSwitch` directive set
- Required `CommonModule`
- Verbose, attribute-based syntax

Modern Control Flow

- Easier to read and scan visually
- Aligns naturally with JavaScript logic
- No module imports required
- Compiler-optimized performance
- Reduced template complexity

Modern control flow simplifies template development while improving both maintainability and runtime performance.



Conditional Rendering with `@if`

The `@if` block replaces `*ngIf` and lets you conditionally include a section of your template when an expression evaluates to truthy. It reads naturally, like an `if` statement in TypeScript — because it is.

Common Use Cases



Login Status

Show authenticated content only when a user is logged in.



User Permissions

Gate UI elements behind role or permission checks.



Error Messages

Display validation or server error feedback contextually.



Loading Indicators

Render spinners or skeletons while async data is pending.

Syntax Example

```
@if (isLoggedIn) {
 <app-dashboard />
}

@if (hasAdminRole) {
 <app-admin-panel />
}

@if (isLoading) {
 <app-spinner />
}

@if (featureFlags.newUI) {
 <app-new-feature />
}
```

Alternative Rendering with `@else`

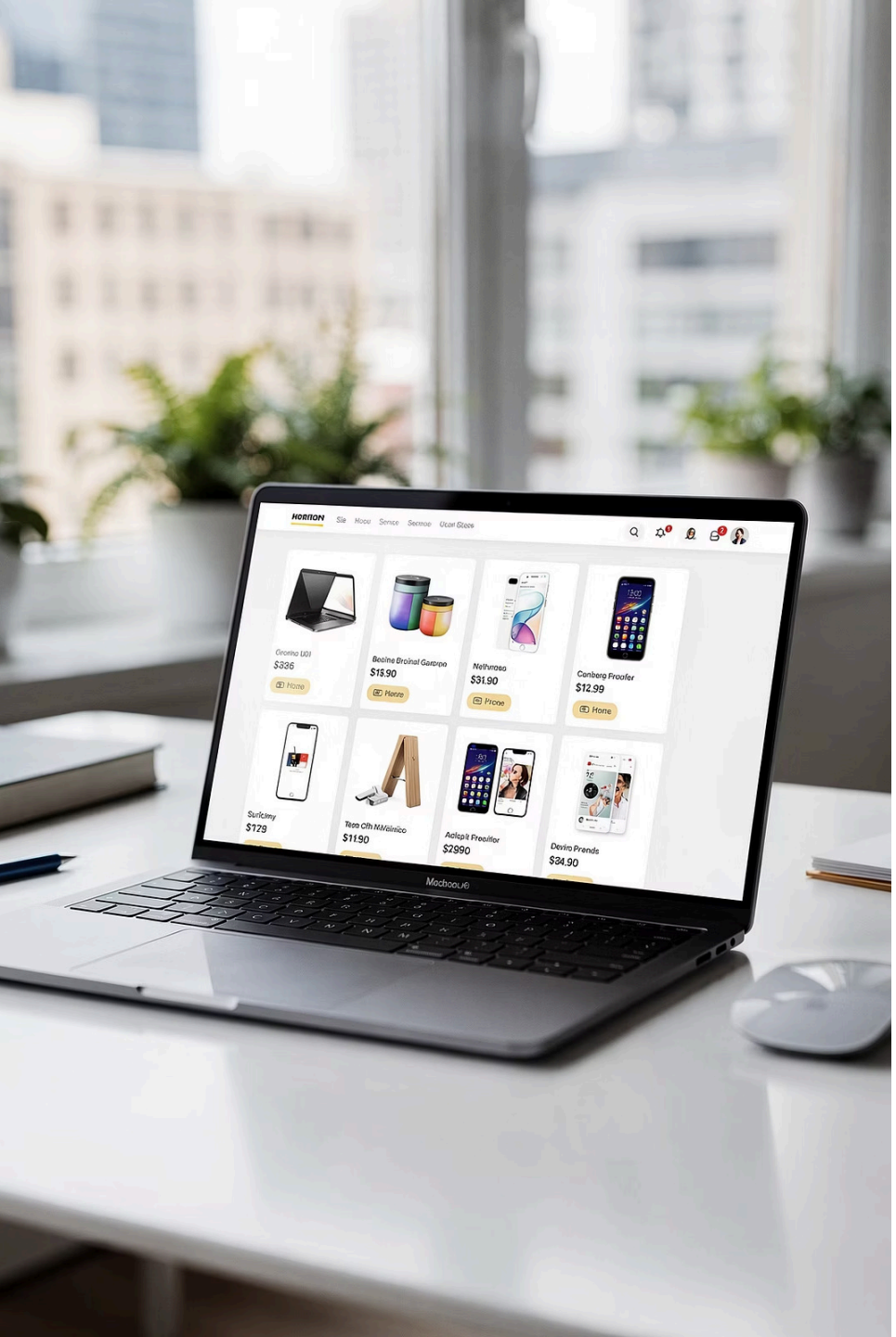
The `@else` block pairs directly with `@if` to provide a fallback view when the condition is false. This eliminates the need for the old `ng-template` + `#ref` pattern that made `*ngIf` with else notoriously verbose.

Login / Logout Show "Welcome back" for authenticated users, a sign-in prompt for guests.	**Data / Empty State** Render results when data is available; show an empty-state message otherwise.
Success / Error Display a success banner on completion, or an error alert on failure.	**Loading / Loaded** Show a skeleton loader during fetch, then reveal real content when ready.

Syntax Example

```
@if (isLoggedIn) {
  <app-user-dashboard />
} @else {
  <app-login-prompt />
}
```

This compact, readable pattern reduces nesting, improves template clarity, and is far more approachable than the legacy `*ngIf="condition; else tmplRef"` pattern.



Iteration Using `@for`

The `@for` block replaces `*ngFor` and iterates over any iterable collection, rendering a template for each item. A required `track` expression helps Angular efficiently identify which DOM nodes to update, add, or remove.

Common Use Cases

- Employee or user lists
- Product catalogs and grids
- Order and transaction histories
- Notification feeds
- Navigation menu items
- Dashboard data tables

Syntax Example

```
@for (product of products;
    track product.id) {
  <app-product-card
    [product]="product" />
}

@for (employee of employees;
    track employee.id) {
  <li>{{ employee.name }}</li>
}
```

Unlike `*ngFor`, the `track` expression is **required** in `@for` — Angular enforces efficient rendering by design.

Tracking Strategies

When rendering collections, Angular needs a way to identify which items have changed between renders. The `track` expression in `@for` tells Angular's diffing algorithm exactly how to match old DOM nodes to new data — directly impacting rendering performance.

1

Track by Unique ID

Recommended. Use a stable database or UUID identifier. Angular performs the most targeted DOM updates possible.

```
track item.id
```

2

Track by Index

Use when items have no unique identifier. Suitable for static or rarely mutated lists, but less efficient for reordering.

```
track $index
```

3

Track by Reference

Use with immutable objects when identity equality is guaranteed. Works well with NgRx or signal-based state.

```
track item
```

3x

Faster Updates

Proper tracking can reduce DOM operations by up to 3× in large lists.

0

Unnecessary Re-renders

Unchanged items are never touched when tracked correctly.

Conditional Branching with `@switch`

The `@switch` block evaluates a single expression and renders the matching `@case` branch — or an `@default` fallback if no case matches. It's the clean replacement for the verbose `[ngSwitch]` + `*ngSwitchCase` directive combination.

Common Scenarios

User Roles

Admin, Editor, Viewer

Order Status

Pending, Shipped, Delivered

Payment Status

Paid, Unpaid, Refunded

App Theme

Light, Dark, High Contrast

Syntax Example

```
@switch (user.role) {
 @case ('admin') {
 <app-admin-view />
 }
 @case ('editor') {
 <app-editor-view />
 }
 @case ('viewer') {
 <app-viewer-view />
 }
 @default {
 <app-guest-view />
 }
}
```

Choosing the Right Control Flow

Selecting the appropriate control flow construct keeps your templates minimal and intent-driven. Each construct maps to a clearly defined use case — use the table below as a quick decision guide during development.

Scenario	Recommended Syntax	Replaces
Single condition	`@if`	`*ngIf`
Condition with fallback	`@if` + `@else`	`*ngIf; else tmpl`
Multiple discrete conditions	`@switch` / `@case`	`[ngSwitch]`
Display a collection	`@for` with `track`	`*ngFor`

Choose the construct that most naturally matches the business requirement. Forcing the wrong tool adds cognitive overhead and makes templates harder to scan at a glance.

Migration from Legacy Directives

Angular applications built before Angular 17 rely on structural directives from `CommonModule`. While these still work in Angular 21, new development and active refactoring should target the modern control flow syntax. Angular's migration schematics can automate much of this transition.

Legacy — Pre Angular 17

```
<div *ngIf="isAdmin">
  Admin Panel
</div>

<li *ngFor="let item of list">
  {{ item.name }}
</li>

<div [ngSwitch]="status">
  <span *ngSwitchCase="'active'">
    Active
  </span>
</div>
```

Modern — Angular 17+ / 21

```
@if (isAdmin) {
  <app-admin-panel />
}

@for (item of list; track item.id) {
  <li>{{ item.name }}</li>
}

@switch (status) {
  @case ('active') {
    <span>Active</span>
  }
}
```

Run the Angular Migration Schematic

Use `ng generate @angular/core:control-flow` to auto-migrate most templates instantly.

Review & Test

Verify migrated templates in your test suite; edge cases with complex `ng-template` refs may require manual adjustment.

Legacy vs. Modern Angular: Side by Side

The shift from structural directives to modern control flow isn't just syntactic sugar — it represents a fundamental improvement in how Angular templates express intent. Here's a direct comparison across every major construct.

Feature	Legacy Directive	Modern Syntax	Key Improvement
Conditionals	`*ngIf`	`@if / @else`	No `ng-template` required
Iteration	`*ngFor`	`@for`	Enforced `track` for perf
Multi-branch	`[ngSwitch]`	`@switch`	Unified block syntax
Module Import	`CommonModule` required	None required	Works in standalone components
Readability	Attribute-based, verbose	Block-based, concise	Feels like native TypeScript

For Angular 17+ and Angular 21 projects, always prefer the modern control flow syntax — unless actively maintaining a legacy codebase that hasn't been migrated yet.

Best Practices

Following these guidelines ensures your Angular 21 templates remain clean, performant, and easy for the entire team to maintain — whether you're writing new code or refactoring legacy applications.



Keep Conditions Simple

Move complex logic into component methods or getters. Templates should read like plain English, not TypeScript expressions.



Always Track in @for

Never skip the `track` expression. Use a unique ID whenever available — fall back to `$index` only when necessary.



Write for Readability

Your future self and teammates will read this template more than you'll write it. Optimize for comprehension, not brevity.



Avoid Deep Nesting

Deeply nested `@if` inside `@for` inside another `@if` signals a component that needs to be split up.



Use @switch for Multiple Branches

Chained `@if` / `@else if` blocks are harder to scan. Reach for `@switch` the moment you have three or more conditions.



Follow Angular 21 Standards

Align with the official Angular style guide. Use standalone components and modern control flow together for a fully modern app architecture.

Module Summary

You've completed Module 3. Here's everything you covered — and what comes next.

What We Covered

01

Modern Control Flow

Angular 17+'s block-based template syntax and why it supersedes structural directives.

02

`@if` and `@else`

Declarative conditional rendering with clean, readable fallback blocks.

03

`@for` with Tracking

Efficient collection rendering backed by mandatory track expressions.

04

`@switch` Branching

Multi-condition logic that scales without deeply nested conditionals.

05

Migration & Best Practices

Migrating legacy directives and Angular 21 template standards.

Up Next: Module 4

> Build an **Employee Portal** using Angular 21 with modern control flow and standalone components.

You'll apply everything from this module in a hands-on project — rendering employee lists with `@for`, gating admin panels with `@if`, and switching department views with `@switch`. All built with fully standalone, `CommonModule`-free components.

- **Standalone Components**
- **Modern Control Flow**
- **Real-World Patterns**